1. Overview

1.1 Purpose

pyquick is a Python library that exposes QUICK's quantum chemistry calculations through a Python interface backed by f2py Fortran bindings. Its primary goal is to automate large-scale molecular dataset generation for machine learning (ML) workflows in computational chemistry and drug design.

1.2 Scope

This document covers the design decisions, API surface, data schemas, output formats, and testing strategy for the initial release of pyquick. It does not cover GUI tooling, cloud deployment, or SLURM job submission.

1.3 Relationship to Existing Work

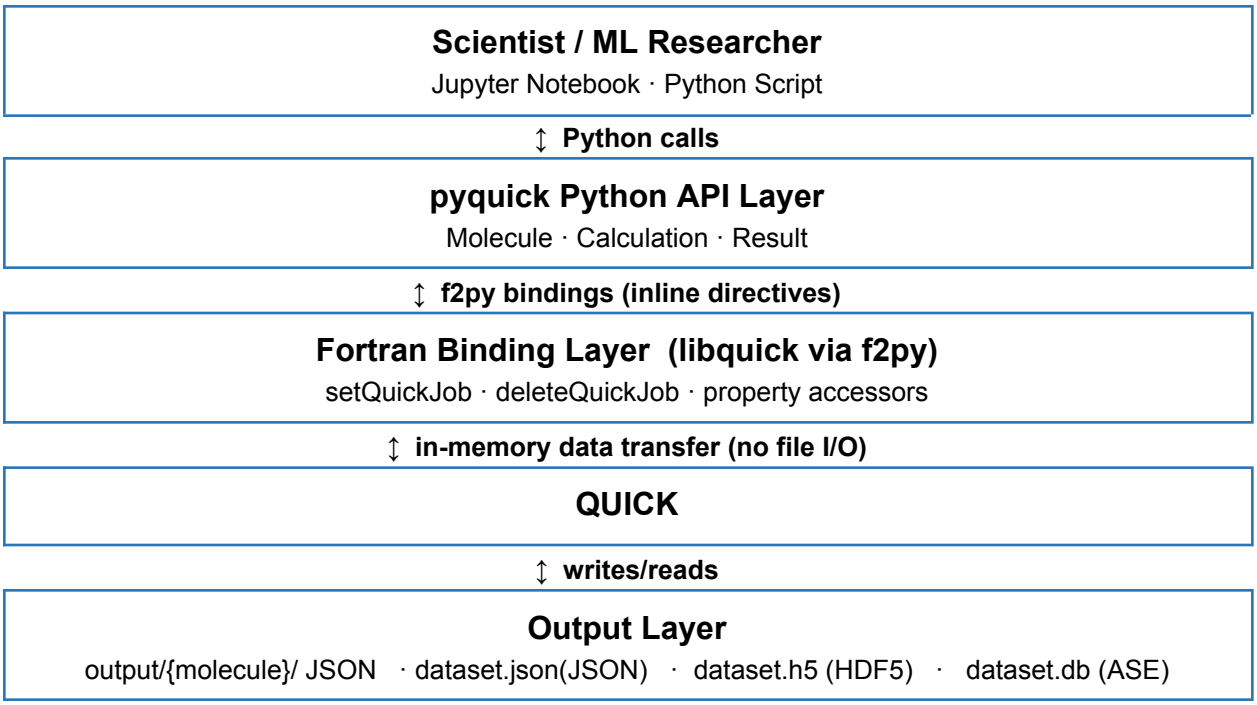
The existing QUICK AMBER API exposes three core subroutines (setQuickJob, getQuickEnergyGradients, and deleteQuickJob) and serves as the primary internal reference for pyquick's binding layer. pyquick wraps and extends that API, making it directly accessible from Python. The design is also informed by the PySCF's property access conventions.

2. Architecture

2.1 Three Layer Overview

- Fortran binding layer — f2py wrappers around QUICK's to expose setQuickJob, getQuickEnergy, getQuickGradients, deleteQuickJob, and additional property accessors as Python-callable functions. (check main.f90)
- Python core layer — the main user-facing classes (Molecule, Calculation, Result, BatchRunner) that wrap the bindings, manage computation state, handle errors, and format outputs.
- Output layer — serializers that write per-molecule JSON, a combined JSON dataset, HDF5 dataset, and an ASE database at batch completion.

2.2 Architecture Diagram



2.3 Key Design Principles

- No required knowledge of Fortran, MPI, or CUDA from the user because the API should abstract all setup. (MPI/CUDA will be in future work, not prioritized in the current phase)
- Crash-safe batch runs: per-molecule HDF5 files are written incrementally so a failed batch can be resumed by re-running (existing files are skipped).
- Use check() directives for error handling
- Properties are accessed as attributes or method calls on the Result object, not parsed from text files.

3. Core API Classes

3.1 Molecule

Represents a molecular structure.

```
water_mol = Molecule.from_file("water.xyz")
# Or
Water_mol = [....] # follow the current input format

# or read a batch from HDF5 file

# or import RDKit to generate input format like
from rdkit import Chem
```

```
from rdkit.Chem import AllChem

mol = Chem.MolFromSmiles("CCO")         # parse ethanol from a SMILES string
mol = Chem.AddHs(mol)                   # QC needs explicit hydrogens
AllChem.EmbedMolecule(mol)             # generate a 3D conformer (coordinates)
AllChem.MMFFOptimizeMolecule(mol)      # quick cleanup of the geometry

conf = mol.GetConformer()
for atom in mol.GetAtoms():
    pos = conf.GetAtomPosition(atom.GetIdx())
    print(atom.GetSymbol(), pos.x, pos.y, pos.z)   # element + xyz → feed to
QUICK
```

3.2 Calculation

Holds computation settings. Set error message for incomplete setting.

```
calc = Calculation(
   functional = "B3LYP",    basis_set  = "6-31G*",
   properties = ["energy", "dipole"])
```

3.3 Result

Returned by a completed calculation. Properties are accessed as attributes. Raises AttributeError if a property was not requested in Calculation.properties.

```
result = calc.run(mol)
print(result.energy)            # float, Hartree
print(result.forces)
result.to_single_json("output/water/result.json")
```

3.4 BatchRunner

Runs a Calculation over a list of Molecule objects. Writes per-molecule JSON as each job completes. Skips molecules whose output JSON already exists, enabling crash-safe resume.

```
from pyquick import BatchRunner
runner  = BatchRunner(
   calculation = calc,
   output_dir  = "output/",     # writes output/{molecule_name}/result.json)
dataset = runner.run(molecules)  # blocks until all molecules are done
dataset.to_json("dataset.json")    # combined JSON
dataset.to_hdf5("dataset.h5")    # combined HDF5
dataset.to_ase_db("dataset.db")  # combined ASE database
```

4. Supported Properties

total SCF energy - ETot

core (nuclear repulsion) energy - ECore

total electronic energy - EEL

one-electron energy - E1e

exchange-correlation energy - Exc

dispersion correction energy - Edisp

mulliken charges - Mulliken(natom)

lowdin charges - Lowdin(natom)

Dipole → not exposed

Alpha molecular orbital energy - E(NBSuse)

Gradient - gradient(3*natom)

Beta MO energies - Eb(NBSuse)

Beta density matrix - denseb(nbasis,nbasis)

Alpha density matrix - dense(nbasis,nbasis)

MP2 correlation energy - EMP2

5. Output Files

HDF5 is written incrementally per molecule during a batch run.

Setup the work directory for output files

Written to output/{molecule_name}/result.HDF5 after each molecule completes. If the file already exists when the batch runner starts, that molecule is skipped, enabling crash-safe resume. Intended for debugging and per-molecule inspection.

```
{
"molecule": { "name": "aspirin", "n_atoms": 21, "atomic_numbers": [6, 6, 8,
...], "positions": [[x, y, z], ...], "charge": 0, "multiplicity": 1},
"settings": { "functional": "B3LYP", "basis_set":  "6-31G*", "scf_conv":
1e-8},
"properties": { "energy": -648.0123456789, "forces": [[fx, fy, fz], ...]},
"metadata": { "pyquick_version": "0.1.0", "quick_version": "25.03",
"timestamp": "2026-06-01T14:32:00Z", "wall_time_s": 12.4}
```

```
}
```

QUICK already writes Molden (quick_api_module.f90). Surface it in the API.

6. User-Facing API Examples

6.1 Single Molecule Calculation

```
mol    = Molecule.from_file("water.xyz")
calc   = Calculation(functional="B3LYP", basis_set="6-31G*",
   properties=["energy", "mulliken_charges", "dipole"])
result = calc.run(mol)
print(result.energy)            # -76.4...
print(result.mulliken_charges)  # [-0.83, 0.41, 0.41]
print(result.dipole)            # [0.0, 0.0, 1.85]
# if there is any basic setting missing, give error message
```

6.2 Batch Run from a YAML Config

Batch usage for dataset generation. Users prepares a YAML file listing all molecules and settings; pyquick handles everything else.

```
# batch_config.yaml
settings:
  functional: B3LYP
  basis_set: 6-31G*
  properties:
    - energy
    - forces
    - esp_charges
    - mulliken_charges
    - dipole
molecules:
  - name: water
    file: molecules/water.xyz
  - name: caffeine
    file: molecules/caffeine.xyz
```

```
from pyquick import BatchRunner
runner = BatchRunner.from_yaml("batch_config.yaml",output_dir="output/",
   n_workers=8)  # optional override multiprocessing worker amount
Dataset =runner.run()
```

```
dataset.to_hdf5("dataset.h5")
dataset.to_ase_db("dataset.db")
```

6.3 Running on a Cluster after MPI/GPU Enabled

pyquick works inside any SLURM job script. Users write the SLURM script themselves.

```
#!/bin/bash
#SBATCH --job-name=pyquick_batch
#SBATCH --time=08:00:00
#SBATCH --gres=gpu:1
#SBATCH --ntasks=4
python run_batch.py    # calls BatchRunner internally
```

7. Testing Strategy

- pyquick output must match QUICK's own reference values
- pytest unit test for each function
- e2e test on 3-5 small molecules and validate the output by using it in ML framework
- Cross-Validation Against PySCF and Psi4

8. Future Work

MPI/GPU/Multi-GPU enable